Functions no PostgreSQL (PL/pgSQL)

Material de Apoio Teórico e Prático — Banco de Dados

🎥 **Videoaula Explicativa:** Este PDF serve como documentação de apoio. Para esclarecimentos detalhados de cada função, assista ao vídeo no YouTube: **https://youtu.be/87P4JPbdf-U**

📌 **Nota Contextual:** Este material é uma continuação direta das atividades realizadas em aula no banco de dados `restaurante` (criação das tabelas `mesa`, `funcionario`, `produto`, `venda`, `itens_venda` e `comissao`, povoamento e consultas SQL).

1. O QUE É UMA FUNCTION EM POSTGRESQL?

Uma **Function** (ou Stored Procedure com retorno) é um bloco de código SQL e PL/pgSQL (*Procedural Language/PostgreSQL Structured Query Language*) armazenado diretamente no servidor de banco de dados. Elas permitem estruturar lógica de programação — como variáveis, estruturas condicionais (`IF/ELSE`) e laços de repetição (`LOOP`) — para executar operações complexas no próprio banco.

2. VANTAGENS E DESVANTAGENS

Vantagens	Desvantagens
Desempenho: Reduz o tráfego de rede, executando a lógica diretamente no servidor onde os dados residem.	**Portabilidade Limitada:** Código escrito em PL/pgSQL não é compatível com outros SGBDs (ex: MySQL, Oracle, SQL Server).
Reutilização e Centralização: Regras de negócio ficam centralizadas no banco, podendo ser chamadas por qualquer aplicação.	**Consumo de CPU no SGBD:** Processamentos muito intensos podem sobrecarregar o servidor do banco de dados.
Segurança: Usuários podem receber permissão para executar a função sem ter acesso direto às tabelas base.	**Manutenção e Versionamento:** O depuramento (debug) e versionamento via Git demandam maior controle.

3. FUNÇÃO 1: CONSULTA COM REGRA DE NEGÓCIO (RETORNO TEXT)

Verifica o status do funcionário e trata a ausência do registro através do comando `FOUND`.

```
 CREATE OR REPLACE FUNCTION retorna_nome_funcionario(p_func_id INT)
RETURNS TEXT AS $$
DECLARE
    v_nome  TEXT;
    v_ativo BOOLEAN;
BEGIN
    SELECT funcionario_nome, funcionario_ativo
      INTO v_nome, v_ativo
      FROM funcionario
     WHERE funcionario_id = p_func_id;

    IF NOT FOUND THEN
        RETURN 'Funcionário não encontrado!';
    END IF;

    IF v_ativo THEN
        RETURN v_nome || ' - Funcionário Ativo';
    ELSE
        RETURN v_nome || ' - Funcionário Inativo';
    END IF;
END;
$$ LANGUAGE plpgsql;

-- Exemplos de Execução:
SELECT retorna_nome_funcionario(1); -- Funcionário encontrado
SELECT retorna_nome_funcionario(99); -- Trata id inexistente
SELECT f.funcionario_codigo, retorna_nome_funcionario(f.funcionario_id) FROM funcionario f;
```

4. FUNÇÃO 2: RETORNO ESCALAR E TRATAMENTO DE NULOS (NUMERIC)

Retorna o percentual de comissão do funcionário utilizando `COALESCE` para tratar nulos ou não encontrados.

```
 CREATE OR REPLACE FUNCTION rt_valor_comissao(p_func_id INT)
RETURNS NUMERIC(5, 2) AS $$
DECLARE
    v_valor_comissao NUMERIC(5, 2);
BEGIN
    SELECT funcionario_comissao INTO v_valor_comissao
      FROM funcionario WHERE funcionario_id = p_func_id;

    RETURN COALESCE(v_valor_comissao, 0.00);
END;
$$ LANGUAGE plpgsql;

-- Uso em Query e Cálculo de Comissão em Reais:
SELECT f.funcionario_nome,
       rt_valor_comissao(f.funcionario_id) AS taxa_pct,
       (100.00 * (rt_valor_comissao(f.funcionario_id) / 100)) AS valor_comissao_reais
  FROM funcionario f WHERE f.funcionario_codigo = 'FUNC-002';
```

5. FUNÇÃO 3: PROCESSAMENTO EM LOTE COM CURSOR E LOOP (VOID)

Percorre vendas não comissionadas em um intervalo de datas, utiliza **Cursor parametrizado** e realiza a inserção do pagamento na tabela `comissao` com atualização do status da venda.

```
ALTER TABLE venda ADD COLUMN IF NOT EXISTS venda_comissionado BOOLEAN DEFAULT FALSE;

CREATE OR REPLACE FUNCTION calc_comissao(data_ini TIMESTAMP, data_fim TIMESTAMP)
RETURNS VOID AS $$
DECLARE
    total_comissao  NUMERIC(10, 2) := 0;
    porc_comissao   NUMERIC(5, 2)  := 0;
    reg             RECORD;

    cr_porce CURSOR (p_func_id INTEGER) IS
        SELECT rt_valor_comissao(p_func_id) AS perc;
BEGIN
    FOR reg IN (
         SELECT venda_id, funcionario_id, venda_total
           FROM venda
          WHERE venda_data_criacao >= data_ini
            AND venda_data_criacao <= data_fim
            AND (venda_comissionado IS FALSE OR venda_comissionado IS NULL)
    ) LOOP
        OPEN cr_porce(reg.funcionario_id);
        FETCH cr_porce INTO porc_comissao;
        CLOSE cr_porce;

        total_comissao := (reg.venda_total * porc_comissao) / 100;

        INSERT INTO comissao (
            funcionario_id, comissao_valor, comissao_ativo,
            comissao_data_criacao, comissao_data_atualizacao
        ) VALUES (
            reg.funcionario_id, total_comissao, TRUE, NOW(), NOW()
        );

        UPDATE venda SET venda_comissionado = TRUE WHERE venda_id = reg.venda_id;

        total_comissao := 0;
        porc_comissao  := 0;
    END LOOP;
END;
$$ LANGUAGE plpgsql;
```

6. EXECUÇÃO E VALIDAÇÃO DAS FUNCTIONS NA BASE RESTAURANTE

```
-- 1. Executando a Function de Cálculo de Comissão para o ano letivo
SELECT calc_comissao('2026-01-01 00:00:00', '2026-12-31 23:59:59');

-- 2. Conferindo os resultados gerados na base de dados
SELECT * FROM comissao;
SELECT venda_id, venda_codigo, venda_comissionado FROM venda;
```

Trabalho 02 - Funções - Gestão de Banco de Dados

Atividade Avaliativa Prática — Entrega Obrigatória no GitHub

🚀 Próxima Etapa do Projeto: Functions no PostgreSQL!

Atenção alunos: Esta é uma atividade prática que deverá ser desenvolvida individualmente e submetida para avaliação. Agora que o seu banco de dados de tema livre já tem estrutura, dados e consultas funcionando, chegou a hora de dar inteligência ao sistema!

🎯 O que fazer:

- Crie **no mínimo 3 (três) Functions** aplicadas ao cenário do seu projeto.
- Pense em utilidades reais para o seu tema: cálculos automáticos, validação de regras de negócio, rotinas de atualização ou relatórios resumidos. As funções precisam fazer sentido no contexto do seu sistema!

📁 Onde entregar:

- No GitHub de vocês (`https://github.com/ra00xxxx-lib` — *Obs: xxxx = número do seu RA*)
- **Repositório:** `bimestre02-2026`
- **Pasta:** `trabalho02`
- Envio do arquivo `.sql` com todas as funções.

▶ **Dúvidas sobre a utilização do GitHub?** Assista ao vídeo instrucional:

https://youtu.be/fTQV9tFenJg?si=y_eECHlqVq_POuVR

✉ **Dúvidas sobre as regras do trabalho?** Envie um e-mail para:
julianosilva@libertas.edu.br